

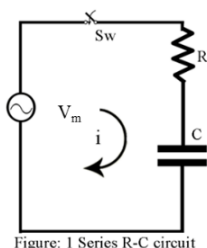
Ordinary Differential Equation: IVP

Part 1

Example Problem

Problem

Solve for the response of an RC circuit with a sinusoidal input



$$f(t, v) = \frac{dv}{dt} = -\frac{1}{\tau} v(t) + \frac{1}{\tau} V_m \cos(2\pi f t)$$

$$\tau = 1; T = 1/\tau; f = 10; V_m = 1; w = 2\pi f;$$

The RC function is defined in @myRC(t,v)

```
function dvdx = myRC(t,v)
    tau=0.01; T=1/tau; f=100; Vm=1;
    dvdx = -T*v + 1*T*Vm*cos(2*pi*f*t+pi/4);
```

Parameter Definitions

```
clear
close all
a=0; b=0.1;
h=0.001;
yINI = 0;
t=a:h:b;
y(1) = yINI;
N = (b-a)/h;
```

True Solution

$$v(t) = \frac{V_m}{\sqrt{1+(\omega\tau)^2}} \left(-\cos(\alpha) e^{-\frac{t}{\tau}} + \cos(\omega t + \alpha) \right), \quad \alpha = -\tan^{-1}(\omega\tau)$$

```
tau=0.01; T=1/tau; f=100; Vm=1; w=2*pi*f;
A=Vm/(sqrt(1+(w*tau)^2));
alpha=-atan(w*tau);
y=-A*cos(alpha+pi/4)*exp(-t/tau)+A*cos(w*t+alpha+pi/4);
```

MATLAB Solution : ode45()

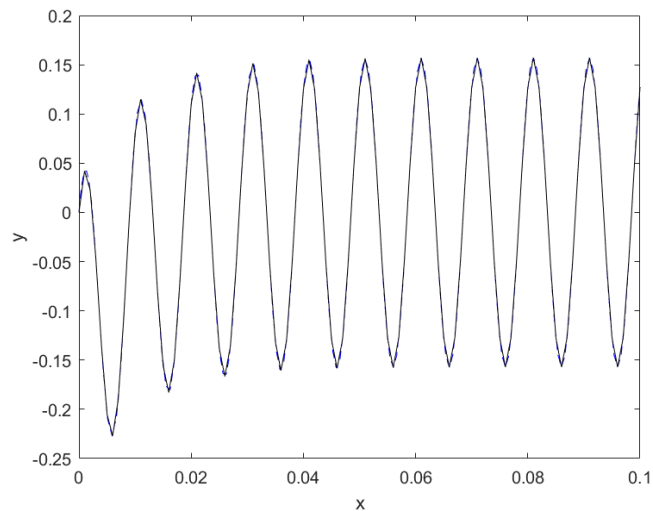
```
% MATLAB's function ODE45
```

```
[tmat,ymat] = ode45(@myRC, [a b], yINI); % Fourth/Fifth RK
```

```
figure()
```

```
plot(tmat,ymat,'--b',t,y,'k')
```

```
xlabel('x'); ylabel('y')
```



Numerical Solution

Euler's explicit method

```
yE(1) = yINI;
```

```
for i = 1:N
```

```
    t(i+1) = t(i) + h;
```

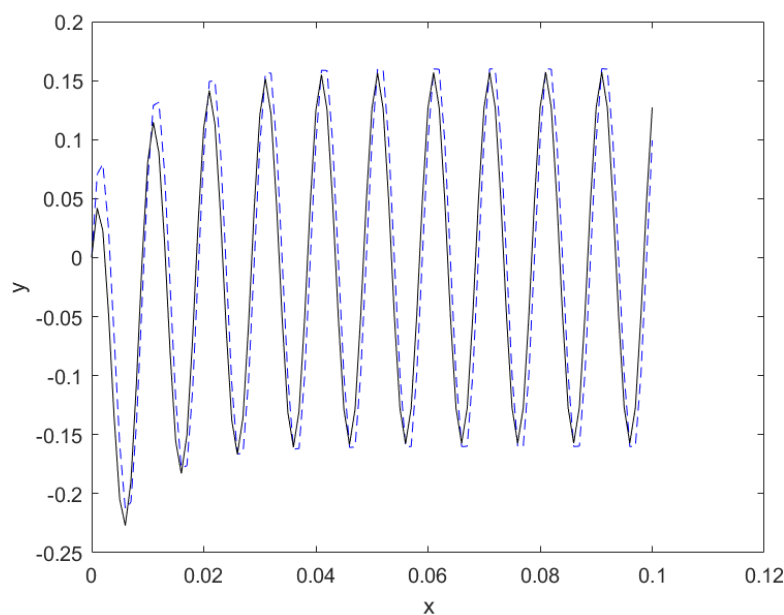
```
    yE(i+1) = yE(i) + myRC(t(i),yE(i))*h;
```

```
end
```

```
figure()
```

```
plot(t,yE,'--b',t,y,'k')
```

```
xlabel('x'); ylabel('y')
```

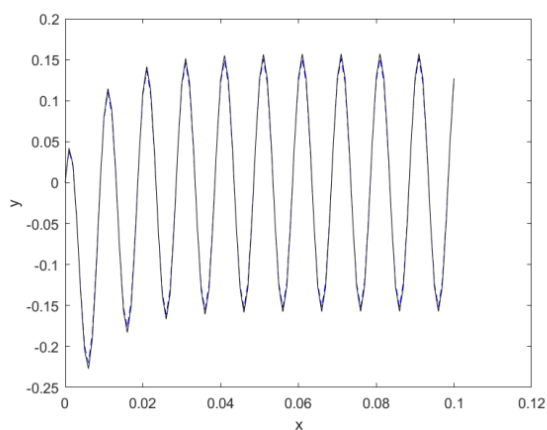


Euler's Modified method

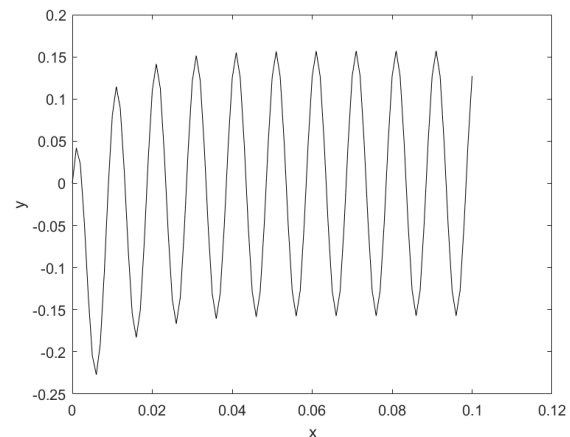
```
yEu = 0;  
Slope2=0;  
yEM(1) = yINI;  
for i = 1:N  
    t(i+1) = t(i) + h;  
    Slope1 = myRC(t(i),yEM(i));  
    yEu = yEM(i)+Slope1*h;  
    Slope2= myRC(t(i+1), yEu);  
    yEM(i+1) = yEM(i) + (Slope1+Slope2)*h/2;  
end  
figure()  
plot(t,yEM,'--b',t,y,'k')  
xlabel('x'); ylabel('y')
```

Runge-Kutta 4th order

```
t0=a; tf=b;  
%[xrk, yrk] = odeRK2(@myRC,t0,tf,h,yINI); % 2nd RK  
[trk, yrk] = odeRK4(@myRC,t0,tf,h,yINI); % 4th RK  
  
figure()  
plot(trk,yrk,'--b',t,y,'k')  
xlabel('x'); ylabel('y')
```



(Euler's modified)



(Runge-Kutta 4th order)

Lecture Objective

This lecture will cover how to solve an initial value problem ODE(ordinary differential equation). First, we will cover Systems of 1st order ODEs. Then, we will continue with solving a higher-order ODE. The numerical methods that we will learn include:

- Euler's method
- Modified Euler's method
- Runge-Kutta Methods (2nd~4th order)

Introduction of Differential Equation

A differential equation is an equation that contains derivatives of an unknown function. If the differential equation has **one** independent variable, then it is called an Ordinary differential equation (ODE).

The first-order ODE has the form of

$$\frac{dy}{dt} = f(t, y(t))$$

Examples of ODEs

Ex1) $\frac{dy}{dt} = ay(t),$

Ex1) $\dot{\mathbf{y}}(t) = \mathbf{A}\mathbf{y}(t)$

Ex 2) $\frac{dy}{dt} = ay(t) + u(t)$

Ex 2) $\dot{\mathbf{y}}(t) = \mathbf{A}\mathbf{y}(t) + \mathbf{u}(t)$

$$\dot{v}(t) = -\frac{1}{\tau}v(t) + \frac{1}{\tau}v_{in}(t)$$

$$m \frac{d^2x}{dt^2} + c \frac{dx}{dt} + kx(t) = F(t)$$

The 1st-order ODE is linear if it is defined as a linear function of y and dy/dt

$$\frac{dy}{dt} = ay(t) + bt^2 \quad // \text{ linear}$$

$$\frac{dy}{dt} = ay^2(t) + bt^2 \quad // \text{ non-linear}$$

First-order ODE solutions

There are infinite number of solution functions that satisfy the ODE equation.

- e.g. $y(t) = ce^{at}$, where c is a constant

To obtain a specific solution, the initial condition or constraint must be given

- e.g. $y = y_0$ at $t = 0 \rightarrow c = y_0$

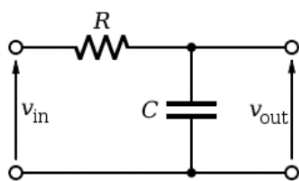
Analytical solution

Example 1: ODE solution using eigenvalue:

$$\dot{y} = \lambda y \rightarrow y = e^{\lambda t} y(0)$$

$$\dot{\mathbf{y}}(t) = \mathbf{A}\mathbf{y}(t) \rightarrow \mathbf{y}(t) = ?$$

Exercise: Solve for the 1st order system of an RC circuit with DC input.



$$\dot{v}(t) = -\frac{1}{\tau} v(t) + \frac{1}{\tau} v_{in}(t)$$

Homogeneous:

$$v(t) = K_0 e^{-t/\tau}, \quad v(0) = K_0$$

Forced response: with DC input V_N ,

$$v_{in}(t) = v_N,$$

$$\rightarrow v(t) = K_1 = v_N$$

Total Solution:

$$v(t) = \underbrace{K_0 \cdot e^{-\frac{t}{\tau}}}_{\text{homogeneous}} + \underbrace{K_1}_{\text{heterogeneous.}}$$

Numerical Solution

In many situations, the analytical solution may not be possible. When using a numerical solution, it needs to define the domain of the solution

- The independent variable x is in the range of $x = a$ to $x = b$.

Depending on the method, the number of points or intervals between $x=[a, b]$ can be set in advance or can be recalculated in each iteration

- N subintervals are used, then $x_1 = a$ to $x_{N+1} = b$

A simple concept of ODE solver:

$$dy = f(x, y)dx$$

$$\int_{y_i}^{y_{i+1}} dy = \int_{x_i}^{x_{i+1}} f(x, y) dx$$

$$\begin{aligned} y_{i+1} &= y_i + \int_{x_i}^{x_{i+1}} f(x, y) dx \\ &\cong y_i + f(x, y) \underbrace{(x_{i+1} - x_i)}_h \end{aligned}$$

Popular ODE methods are

- Euler's method (explicit, implicit, modified)
- Midpoint method
- Runge-Kutta
- Multi-step method
- Predictor-Corrector

*Method depends on which integration method we choose to use.

Q) Compare the above methods in terms of stability and accuracy

- Explicit method : simple but low accuracy & stability
- Implicit method : more complex but more stable
- Modified & Midpoint method : achieve 2nd order accuracy , improved stability.
- Runge-Kutta ; high accuracy & good stability .
- Multi-step
- Predictor-corrector

Overview of 1st ODE numerical methods

- The solution process is incremental
 - Starts at the point where the initial value is given
 - Then using the first point, the solution is determined at the second point, and so on.
- Single step
 - Solution of x_{i+1} from known solution of x_i
- Multi-step
 - Solution of x_{i+1} from $(x_{i-k}, x_{i-(k+1)}, \dots)$
 - Can give a better solution estimation than a single step method
- Explicit
 - Use an explicit formula for calculating dependent variable at the next value of independent variable.

$$\underbrace{y_{i+1}}_{\text{unknown}} = \underbrace{F(x_i, x_{i+1}, y_i)}_{\text{known}}$$

- Implicit
 - Unknown dependent variables on both sides. Use methods such as non-linear solver. Implicit method gives improved accuracy over explicit method but requires more effort

$$\underbrace{y_{i+1}}_{\text{unknown}} = F(x_i, x_{i+1}, \underbrace{y_{i+1}}_{\text{unknown}})$$

- Errors in numerical solution of ODE
 - Round-off error : Depends on the computer, number of digits for floating point etc
 - Truncation error has two parts
 - Local truncation error: Approximating error at each single step
 - Global truncation error (accumulated): Due to accumulation of each step errors

Single-step explicit method

Approximates the solution of y_{i+1} by using the slope

$$x_{i+1} = x_i + h$$

$$y_{i+1} = y_i + (\text{slope}_i)h$$

Starting from the known point of (x_1, y_1)

- If the slope at (x_i, y_i) is used → Euler's method
- If the average slope at (x_i, y_i) and (x_{i+1}, y_{i+1}^{EU}) → Modified Euler's
- If the slope at $\frac{x_i + x_{i+1}}{2}$ is used → Midpoint
- If the slope is a sum of weighted slopes → Runge Kutta

Euler's Explicit Method

Forward Euler Method is a single-step and the simplest method for solving 1st order ODE of

$$f(x, y) = \frac{dy}{dx} \text{ with initial condition of } y(x_0) = y_0$$

It solves for the next point $y(x)$ using the slope at $f(x, y) = \frac{dy}{dx}$ starting from the initial condition, $y(x_0) = y_0$.

Using rectangular integral method

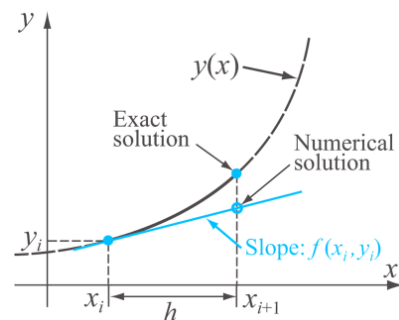
$$y_{i+1} = y_i + \int_{x_i}^{x_{i+1}} f(x, y) dx \cong y_i + f(x_i, y_i)(x_{i+1} - x_i)$$

The estimate value of $y(x)$ at (x_{i+1}, y_{i+1}) is

$$y_{i+1}^N = y_i^N + f(x_i, y_i^N) \cdot h$$

$$x_{i+1} = x_i + h$$

where $f(x_i, y_i) = \left. \frac{dy}{dx} \right|_{x=x_i}$, $h = x_{i+1} - x_i$



Here, the initial condition is known priori, and h is assumed to be a constant value

Derivation of Euler Method:

- Based on numerical integration

$$dy = f(x, y)dx$$

$$\int_{y_i}^{y_{i+1}} dy = \int_{x_i}^{x_{i+1}} f(x, y)dx$$

$$y_{i+1} = y_i + f(x, y) \cdot (x_{i+1} - x_i)$$

- Based on Taylor Series (order 1)

$$y(x+h) = y(x) + y'(x)h + y''(x)\frac{h^2}{2!} + \dots +$$

$$y(x_{i+1}) \approx y(x_i) + y'(x_i)(h)$$

Truncation error of Euler's explicit method

- Global Truncation error $O(h)$: see appendix
- Local Truncation error $O(h^2)$
 - Using Taylor's Series starting from the numerical solution y_i^{NS}

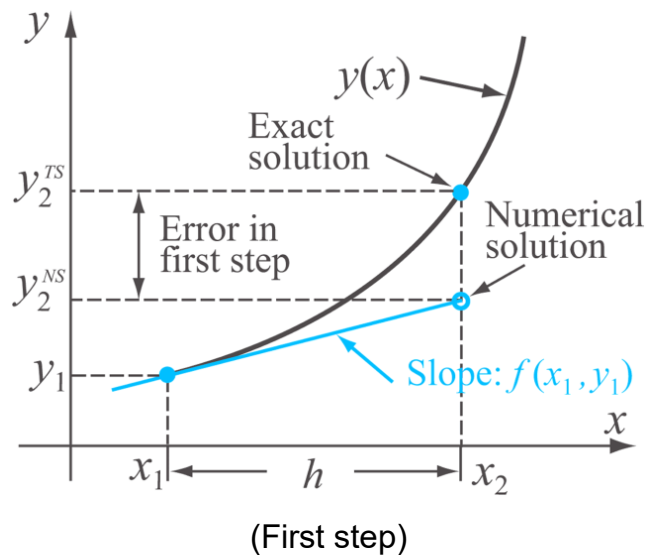
$$y_{i+1}^{Taylor} = y_i^{NS} + f(x_i, y_i^{NS})h + \underbrace{\frac{d^2 y}{dx^2} \Big|_{x=\xi_i} \frac{h^2}{2}}_{O(h^2)}$$

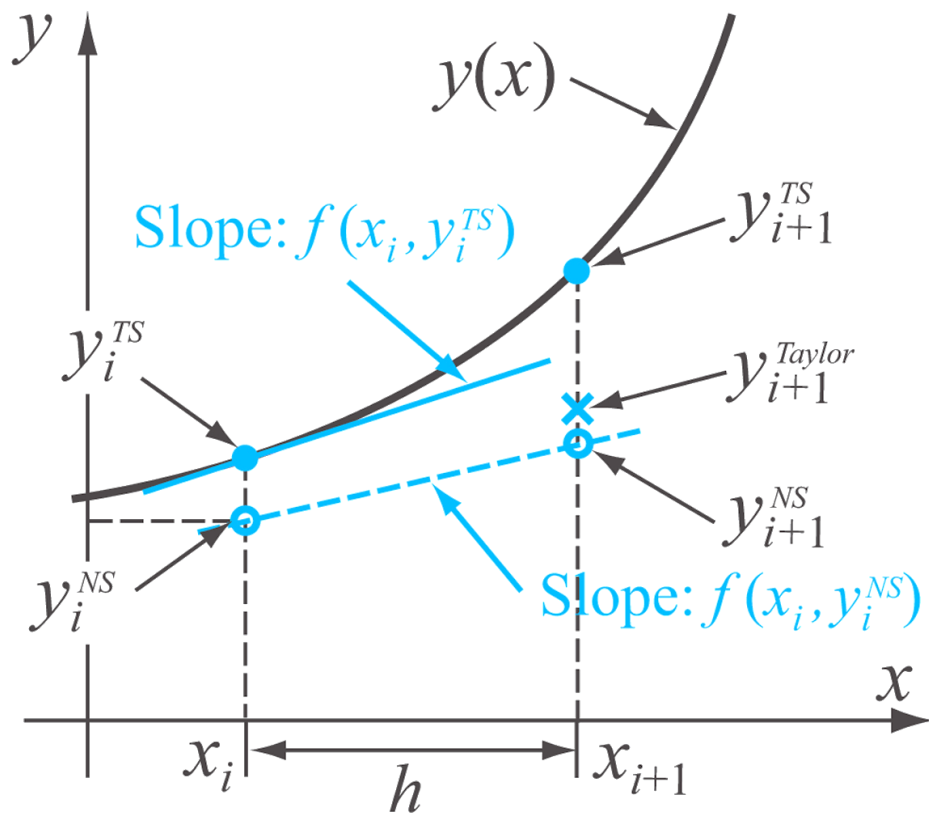
- Since the Euler's method is $y_{i+1}^{NS} = y_i^{NS} + f(x_i, y_i^{NS})h$, the local error of Euler's method is

$$e_i^{TR} = y_{i+1}^{Taylor} - y_{i+1}^{NS} = O(h^2)$$

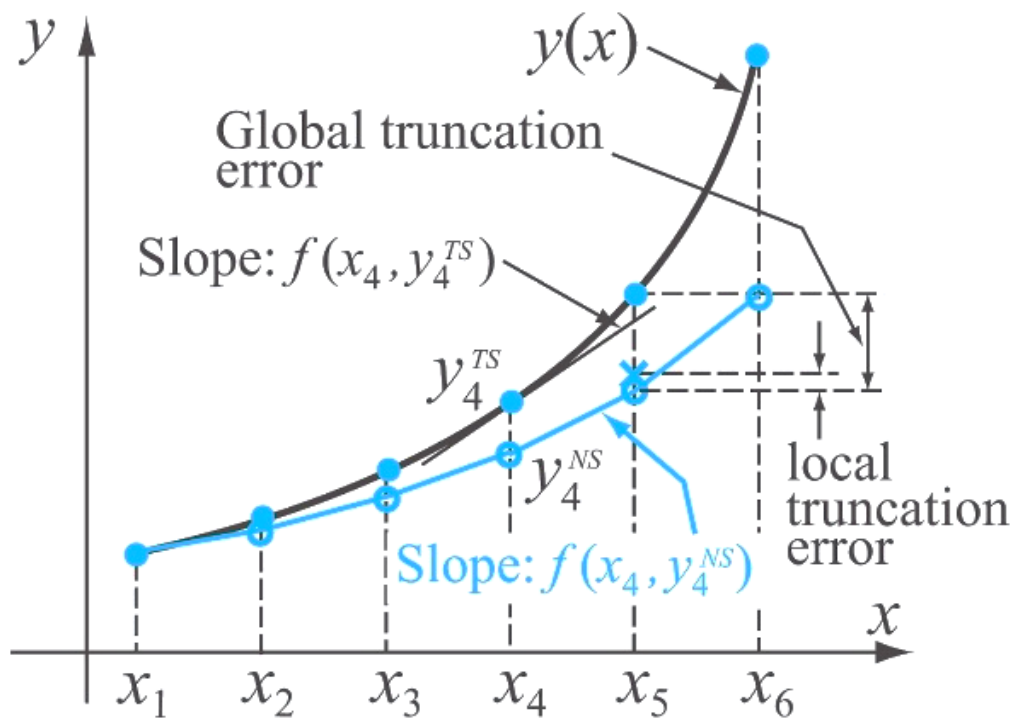
Total truncation error: global error (including local error)

- Global Truncation error $O(h)$: $y^{True} - y^N$
- Local Truncation error $O(h^2)$: $y^{Tay} - y^N$





(After i^{th} step, with accumulated error)



(Local & Global TR error)

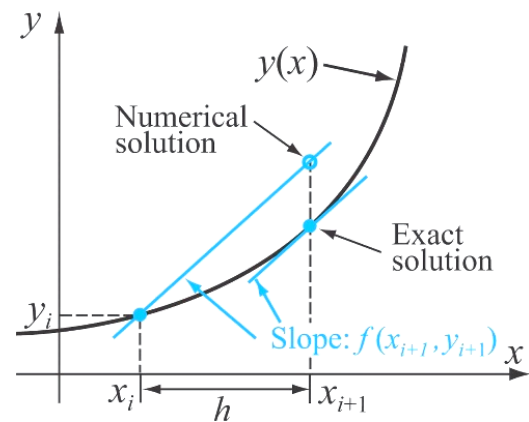
Euler's Implicit Method

Similar to Euler's method but use the slope at unknown (x_{i+1}, y_{i+1}) .

It is also known as backward Euler method and has similar error order as explicit Euler method.

Use any non-linear solver to solve for y^N

$$\underset{\text{unknown}}{y_{i+1}} = F(\underset{\text{unknown}}{x_i}, \underset{\text{unknown}}{x_{i+1}}, \underset{\text{unknown}}{y_{i+1}})$$
$$y^N_{i+1} = y^N_i + f(x_{i+1}, y^N_{i+1})h,$$
$$x_{i+1} = x_i + h$$



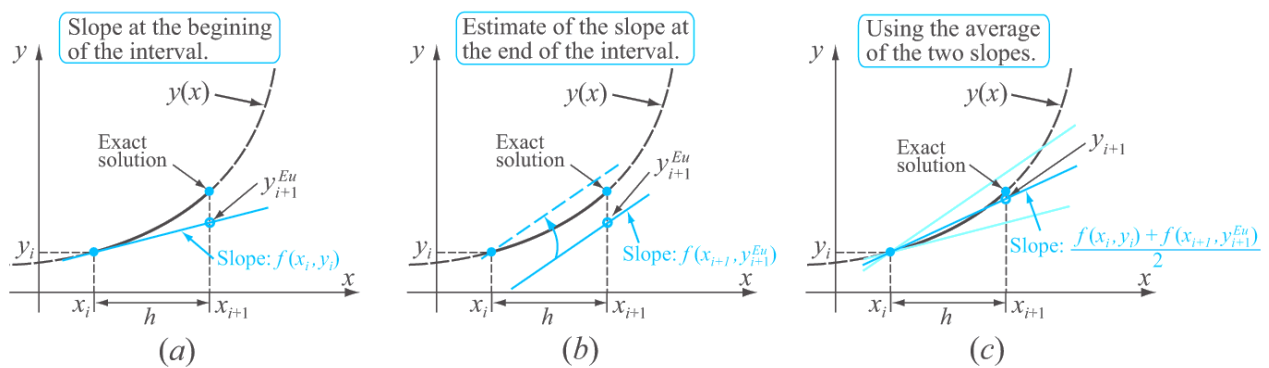
Modified Euler's method

The slope used in this method is the average of the slope at the beginning of the interval and the estimated slope at the end of the interval.

It can be derived by integrating ODE using the trapezoidal method

Using trapezoid integral method

$$y_{i+1}^N = y_i^N + \frac{f(x_i, y_i) + f(x_{i+1}, y_{i+1}^E)}{2} h$$



Process:

- Calculate $\left. \frac{dy}{dx} \right|_{x=x_i} = f(x_i, y_i)$ and calculate $\left. \frac{dy}{dx} \right|_{x=x_{i+1}, y=y_{i+1}^E} = f(x_{i+1}, y_{i+1}^E)$,

$$\text{where } y_{i+1}^E = y_i + f(x_i, y_i)h$$

- Take average slope and get

$$y_{i+1} = y_i + \underbrace{\frac{f(x_i, y_i) + f(x_{i+1}, y_{i+1}^E)}{2}}_{\text{avg slope}} \times h$$

Pseudocode

- Start from the first point of x_0, y_0
- Calculate : $\text{slope} - k_1 = f(x[i], y^N[i])$
- Estimate : $y^E[i+1] = y[i] + f(x[i], y^N[i]) \cdot h$
- update : $x[i+1] = x[i] + h$
- Calculate : $\text{slope} - k_2 = f(x[i+1], y^N[i+1])$

Repeat until the last data point

-

Exercise

Problem 1:

Solve for ODE $f(x, y) = \frac{dy}{dx} = -12y + 7e^{-0.3x}$, from $x=0$ to $x=2.5$.

Let the initial value is $y(0)=3$, $h=0.5$ → 해는 $y(0.5) = 4.07$ 이 나옴

- True analytical solution is $y(x) = \frac{70}{9}e^{-0.3x} - \frac{43}{9}e^{-1.2x}$
- Euler method:

$$y_{i+1}^N = y_i^N + f(x_i, y_i^N)(x_{i+1} - x_i)$$

$h=0.5$

Initially, $x[0]=0$, $y[0]=3$ for $x=0$ to $x=2.5$		index	0	1	2	3	4	5
$y[i+1] = y[i] + f(x[i], y[i])h$ $= y[i] + (-12y[i] + 7e^{-0.3x[i]})h$	x		0	0.5	1.0	1.5	2.0	2.5
$x[i+1] = x[i] + h$	y		3	-11.5				
end				31.76				

이제 차이를
구해서 $h=0.5$

$$\frac{dy}{dx} = f(x, y) = -12y + 7e^{-0.3x}$$

- Modified Euler method:

$$y_{i+1}^E = y_i + f(x_i, y_i) * h$$

$$y_1^E = y_0 + f(x_0, y_0) * h$$

$$= 3 + (-12 \times 3 + 7e^{-0.3 \times 0}) * 0.5$$

$$= 3 - 29 \times 0.5 = 3 - 14.5 = -11.5 = y_1^E$$

$$y_1 = y_0 + \frac{f(x_0, y_0) + f(x_1, y_1^E)}{2} \times 0.5$$

$$= 3 + \frac{-29 + 144.025}{2} \times 0.5 = 31.756$$

$f(x_0, y_0) = -12y_0 + 7e^{-0.3 \times x_0}$

$$= -12 \times 3 + 7e^{-0.3 \times 0}$$

$$= -36 + 7 = -29 : \text{첫 번째 } x_0 > 1$$

$f(x_1, y_1^E) = -12y_1 + 7e^{-0.3 \times x_1}$

$$= -12 \times (-11.5) + 7e^{-0.3 \times 0.5}$$

$$= 138 + 7e^{-0.15}$$

$$= 144.025 : \text{다음 } x_1 > 1$$

$k_1, k_2 : \text{기울기 (slope)}$

$C_1, C_2 : \text{가장자리 비}$

Problem2:

Solve for an RC circuit with sinusoidal input $v_{in}(t) = V_m \cos(2\pi ft)$, for $t=0$ to 1 s.

The ODE of the RC circuit is expressed as

$$f(t, v) = \frac{dv}{dt} = -\frac{1}{\tau}v(t) + \frac{1}{\tau}V_m \cos(2\pi ft) = -v(t) + \cos(62.83t)$$

Let the initial value is $v(0)=0$, $h=0.01$ s, $f=10$ Hz, $V_m=1$ [V], $\tau=1$

- True analytical solution:

$$v(t) = \frac{V_m}{\sqrt{1+(\omega\tau)^2}} \left(\cos(\alpha)e^{-\frac{t}{\tau}} + \cos(\omega t + \alpha) \right), \quad v(t) = y$$

where $\alpha = -\tan^{-1}(\omega\tau)$

- Euler method:

initially, $t[0]=0, v[0]=0$

for $t=0$ to $t=1$

$$v(i+1) = v(i) + f(t(i), v(i)) \cdot h$$

$$= v(i) + (-v(i) + \underbrace{\cos(62.83 \cdot t(i))}_{(1)}) \cdot h$$

$$t(i+1) = t(i)$$

end

- Modified Euler method:

for $t=0$ to $t=1$

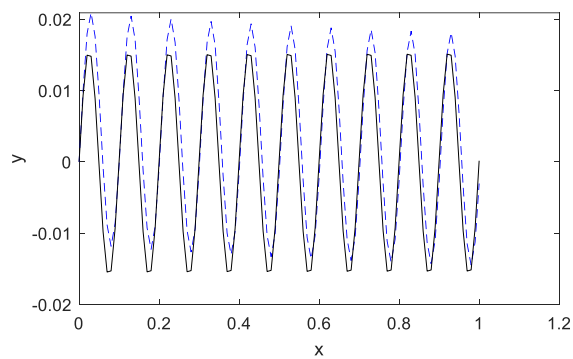
$$k_1 = f(t(i), v(i))$$

$$v(i+1) = v(i) + k_1 \cdot h$$

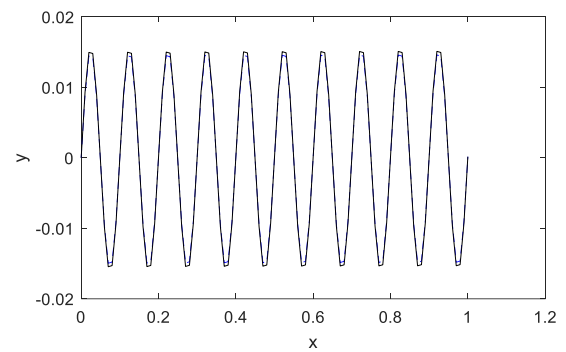
$$k_2 = f(t(i+1), v(i+1))$$

$$v(i+1) = v(i) + (k_1 + k_2) / 2 \cdot h$$

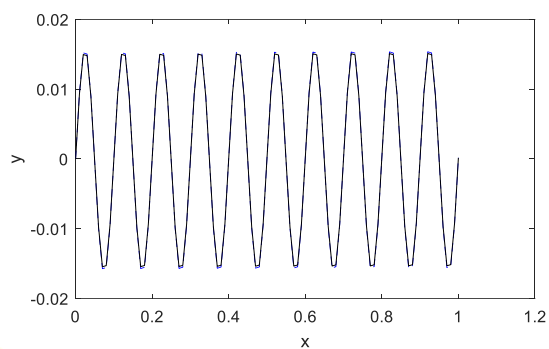
Compare the accuracy of each method



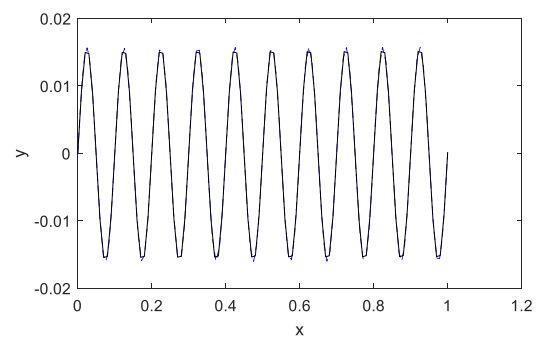
Euler's method



Modified Euler's method



Midpoint Euler method



MATLAB's ode45

```
[xmat,yamat] = ode45(@myRC, [a b], yINI); % Fourth/Fifth RK
figure()
plot(xmat,yamat,'--b',x,y,'k')
xlabel('x'); ylabel('y')
```

Runge-Kutta Methods

It is a family of single-step, explicit NM solution for 1st ODE. The slope is obtained from a weighted sum of gradient at multiple points.

$$y_{i+1} = y_i + slope \cdot h$$

- Second order Runge-Kutta
 - Use slopes at two points with local error $O(h^3)$, Global error $O(h^2)$
- Third order Runge-Kutta
 - Uses slopes at three points with local error $O(h^4)$, Global error $O(h^3)$

Second order Runge-Kutta: 2nd order

$$f(t, y) = \frac{dy}{dt}$$

$$K_1 = f(t_i, y_i),$$

$$K_2 = f(t_i + \alpha h, y_i + \beta K_1 h)$$

$$y_{i+1} = y_i + \overbrace{(C_1 K_1 + C_2 K_2)}^{slope} h$$

$$\Rightarrow y_{i+1} = y_i + C_1 h f(t, y) + C_2 h \left(f(t + \alpha h, y + \beta h f(t, y)) \right) \rightarrow \textcircled{1}$$

Using Taylor's series

$$K_2 = f(t + \alpha h, y + \beta h f) = f(t, y) + \alpha h f_t + \beta h f \cdot f_y + \underbrace{\frac{1}{2} \left[\alpha h \frac{\partial}{\partial t} + \beta h f \frac{\partial}{\partial y} \right]^2 f(\bar{t}, \bar{y})}_{O(h^2)}$$

$$= f + \alpha h f_t + \beta h f \cdot f_y + O(h^2)$$

→ ②

* Note : $f(x+h, y+k) = \sum_{n=0}^{\infty} \frac{1}{n!} \left(h \frac{\partial}{\partial x} + k \frac{\partial}{\partial y} \right)^n f(x, y)$

By substituting ②, ① becomes,

$$\begin{aligned} y_{i+1} &= y_i + (C_1 K_1 + C_2 K_2) h \\ &= y_i + C_1 h f + C_2 h (f + \alpha h f_t + \beta h f \cdot f_y + O(h^2)) \\ &= y_i + (C_1 + C_2) h f + \alpha C_2 h^2 f_t + \beta C_2 h^2 f \cdot f_y + O(h^3) \end{aligned} \rightarrow \textcircled{3}$$

How to find values of C1, C2:

- Write eq ① as : $y_{i+1} = y(t_i + h) = y_i + h f + \frac{1}{2} h^2 \frac{d^2 y}{dt^2} + O(h^3)$

where

$$\begin{aligned} \frac{d^2 y}{dt^2} &= \frac{d}{dt} \left(\frac{d}{dt} y \right) = \frac{d}{dt} f(t, y) \\ &= \frac{\partial f}{\partial t} \frac{dt}{dt} + \frac{\partial f}{\partial y} \frac{dy}{dt} \\ &= f_t + f \cdot f_y \end{aligned}$$

$$y(t+h) = y_{i+1} = y_i + h f + \frac{1}{2} h^2 (f_t + f \cdot f_y) + O(h^3) \rightarrow \textcircled{4}$$

Since ③ is equal to ④

$$\begin{aligned} \Rightarrow y_{i+1} &= y_i + (C_1 + C_2) h f + \alpha C_2 h^2 f_t + \beta C_2 h^2 f \cdot f_y + O(h^3) \\ &= y_i + h f + \frac{1}{2} h^2 f_t + \frac{1}{2} f \cdot f_y + O(h^3) \end{aligned}$$

$$\Rightarrow \begin{cases} C_1 + C_2 = 1 \\ C_2 \alpha = \frac{1}{2} \\ C_2 \beta = \frac{1}{2} \end{cases} \quad 3 \text{ eq. \& 4 variables}$$

To solve for the unique solution, we can set a constraint as $\beta = \alpha$. Then the coefficient C_1 , and C_2 becomes

$$y_{i+1} = y_i + (C_1 K_1 + C_2 K_2)h$$

$$C_1 = 1 - \frac{1}{2\alpha}, \quad C_2 = \frac{1}{2\alpha} \quad \text{and} \quad \begin{cases} K_1 = f(t, y) \\ K_2 = f(t + \alpha h, y + \beta K_1 h) \end{cases}$$

Case 1: Modified Euler's method

$$C_1 = \frac{1}{2}, \quad C_2 = \frac{1}{2}, \quad \alpha = 1, \quad \beta = 1 \quad \rightarrow \quad \begin{matrix} t_{i+1} & y_{i+1}^{\varepsilon} \\ \parallel & \parallel \\ & y_i + K_1 h \end{matrix} \quad \rightarrow \quad K_2 = f(t_i + h, y_i + K_1 h)$$

$$K_1 = f(t_i, y_i)$$

$$\begin{aligned} K_2 &= f(t_i + h, y_i + K_1 h) \\ &= f(t_{i+1}, y_{i+1}^{\varepsilon}) \end{aligned}$$

$$y_{i+1} = y_i + (C_1 K_1 + C_2 K_2) \cdot h$$

$$y(x_{i+1}) = y(x_i) + \left(\frac{1}{2} f(t_i, y_i) + \frac{1}{2} f(t_{i+1}, y_{i+1}^{\varepsilon}) \right) h$$

Case 2 : Midpoint method

$$C_1 = 0, C_2 = 1, \alpha = \frac{1}{2}, \beta = \frac{1}{2}$$

$$K_1 = 0$$

$$K_2 = f\left(\underbrace{x_i + \frac{1}{2}h}_{x_m}, \underbrace{y_i + \frac{1}{2}K_1h}_{y_m^E}\right) \leftarrow \text{Midpoint method}$$

$$y_{i+1} = y_i + K_2h$$

Case 3: Heun's method

$$C_1 = \frac{1}{4}, C_2 = \frac{3}{4}, \alpha = \frac{2}{3}, \beta = \frac{2}{3}$$

$$K_1 = f(x_i, y_i)$$

$$K_2 = f\left(x_i + \frac{2}{3}h, y_i + \frac{2}{3}K_1h\right) \leftarrow \text{Heun's method}$$

$$y_{i+1} = y_i + \left(\frac{1}{4}K_1 + \frac{3}{4}K_2\right)h$$

Third-order Runge-Kutta

$$y_{i+1} = y_i + \overbrace{(C_1 K_1 + C_2 K_2 + C_3 K_3)}^{\text{slope}} h$$

$$\begin{cases} K_1 = f(x_i, y_i) \\ K_2 = f(x_i + \alpha_2 h, y + \beta_{21} K_1 h) \\ K_3 = f(x_i + \alpha_3 h, y_i + \beta_{31} K_1 h + \beta_{32} K_2 h) \end{cases}$$

Case 1 : classical third-order Runge-Kutta

$$C_1 = \frac{1}{6}, C_2 = \frac{4}{6}, C_3 = \frac{1}{6}, \alpha_2 = \frac{1}{2}, \alpha_3 = 1, \beta_{21} = \frac{1}{2}, \beta_{31} = -1, \beta_{32} = 2$$

$$\begin{cases} K_1 = f(x_i, y_i) \\ K_2 = f(x_i + \alpha_2 h, y + \beta_{21} K_1 h) = f\left(x_i + \frac{1}{2}h, y + \frac{1}{2} \cdot f(x_i, y_i)h\right) \\ K_3 = f(x_i + h, y_i - K_1 h + 2K_2 h) \\ \quad = f(x_i + h, y_i - (K_1 - 2K_2) \cdot h) \end{cases}$$

$$y_{i+1} = y_i + \left(\frac{1}{6} \cdot f(x_i, y_i) + \frac{4}{6} \cdot f\left(x_i + \frac{1}{2}h, y + \frac{1}{2} K_1 h\right) + \frac{1}{6} \cdot f(x_i + h, y_i - (K_1 - 2K_2) \cdot h) \right) h$$

Fourth order Runge-Kutta

$$y_{i+1} = y_i + \overbrace{(C_1 K_1 + C_2 K_2 + C_3 K_3 + C_4 K_4)}^{\text{slope}} h$$

$$\begin{cases} K_1 = f(x_i, y_i) \\ K_2 = f(x_i + \alpha_2 h, y + \beta_{21} K_1 h) \\ K_3 = f(x_i + \alpha_3 h, y_i + \beta_{31} K_1 h + \beta_{32} K_2 h) \\ K_4 = f(x_i + \alpha_4 h, y_i + \beta_{41} K_1 h + \beta_{42} K_2 h + \beta_{43} K_3 h) \end{cases}$$

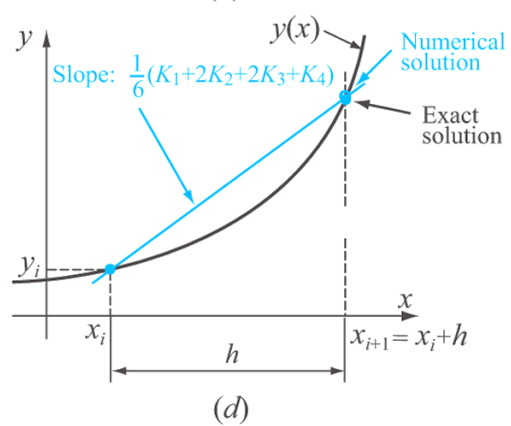
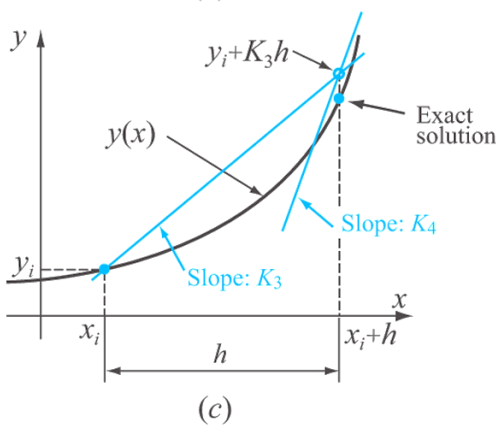
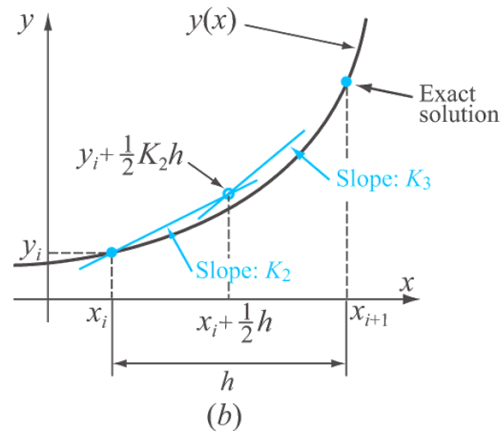
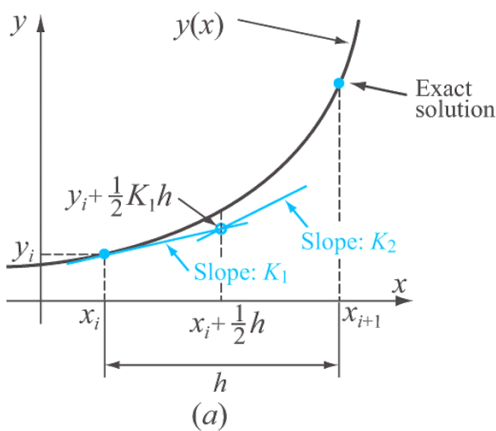
- Classical Fourth-order Runge-Kutta

$$C_1 = C_4 = \frac{1}{6}, \quad C_2 = C_3 = \frac{2}{6}, \quad \alpha_2 = \alpha_3 = \beta_{21} = \beta_{32} = \frac{1}{2},$$

$$\alpha_4 = \beta_{43} = 1, \quad \beta_{31} = \beta_{41} = \beta_{42} = 0$$

$$y_{i+1} = y_i +$$

$$\left\{ \begin{array}{l} K_1 = f(x_i, y_i) \\ K_2 = f\left(x_i + \frac{1}{2}h, y_i + \frac{1}{2}K_1h\right) \\ K_3 = f\left(x_i + \frac{1}{2}h, y_i + \frac{1}{2}K_2h\right) \\ K_4 = f(x_i + h, y_i + K_3h) \end{array} \right.$$



Appendix

Global error of Euler Method: (The total accumulated error)

$$\begin{aligned} E_{N+1}^T &\leq \frac{hM}{C} [e^{hCN} - 1] \quad , \quad N = \frac{x_{N+1} - x_1}{h} \\ &= \frac{hM}{C} [e^{C[x_{N+1} - x_1]} - 1] \Rightarrow \text{order of } h \end{aligned}$$

→ Global Truncation error $O(h)$ (Local Truncation error $O(h^2)$)

Derivation:

$$\begin{aligned} E_i^{TR} &= y_i^{TS} - y_i^{NS} \\ y_{i+1}^{TS} &= y_i^{TS} + f(x_i, y_i^{TS})h + \frac{d^2 y}{dx^2} \bigg|_{x=\eta_i} \frac{h^2}{2} \\ E_{i+1}^{TR} &= y_{i+1}^{TS} - y_{i+1}^{NS} \\ &= y_i^{TS} + f(x_i, y_i^{TS})h + \frac{d^2 y}{dx^2} \bigg|_{x=\eta_i} \frac{h^2}{2} \\ &\quad - (y_i^{NS} + f(x_i, y_i^{NS})h) \\ &= \underbrace{(y_i^{TS} - y_i^{NS})}_{E_i^{TR}} + \underbrace{[f(x_i, y_i^{TS}) - f(x_i, y_i^{NS})]}_{\frac{\partial f}{\partial y} \bigg|_{y=y_i} \underbrace{(y_i^{TS} - y_i^{NS})}_{E_i^{TR}}} h + \frac{d^2 y}{dx^2} \bigg|_{x=\eta_i} \frac{h^2}{2} \\ &= E_i^{TR} + E_i^{TR} (f_y(x_i, y_i)h) + \alpha h^2 \\ &= E_i^{TR} (1 + f_y(x_i, y_i)h) + \alpha h^2 \end{aligned}$$

By mean – value theorem,

$$\begin{aligned} f(x_i, y_i^{TS}) - f(x_i, y_i^N) &= \frac{\partial f}{\partial y} \bigg|_{\substack{y=\gamma_i \\ x=x_i}} (y_i^T - y_i^N) \\ &= E_i^T \frac{\partial f}{\partial y} \bigg|_{\substack{y=\gamma_i \\ x=x_i}} \end{aligned}$$

$$E_{i+1}^T = E_i^T [1 + hf_y(x_i, \gamma_i)] + \alpha_i h^2$$

$$\text{where, } f_y(x_i, \gamma_i) = \frac{\partial f(x, y)}{\partial y} \bigg|_{\substack{y=\gamma_i \\ x=x_i}} \text{ and } \alpha_i = \frac{1}{2} \frac{d^2 y}{dx^2} \bigg|_{x=\eta_i}$$

if $|f_y(x_i, \gamma_i)| \leq C$ bounded, then

$$E_{i+1}^{TR} \leq E_i^{TR} [1 + hC] + \alpha_i h^2$$

at $y_1^T = y_1^{NS}$, initial value $\rightarrow E_1 = 0$

$$E_2^T \leq \alpha_1 h^2$$

$$\begin{aligned} E_3^T &\leq E_2^T [1 + hC] + \alpha_2 h^2 = \alpha_1 h^2 [1 + hC] + \alpha_2 h^2 \\ &= h^2 \{ [1 + hC] \alpha_1 + \alpha_2 \} \end{aligned}$$

$$\begin{aligned} E_4^T &\leq E_3^T [1 + hC] + \alpha_3 h^2 = h^2 \{ [1 + hC] \alpha_1 + \alpha_2 \} [1 + hC] + \alpha_3 h^2 \\ &= h^2 \left[[1 + hC]^2 \alpha_1 + [1 + hC] \alpha_2 + \alpha_3 \right] \end{aligned}$$

$\vdots$

$$E_i^T \leq h^2 \left[[1 + hC]^{i-2} \alpha_1 + [1 + hC]^{i-3} \alpha_2 + \dots + [1 + hC] \alpha_{i-2} + \alpha_{i-1} \right]$$

Substitute $z = 1 + hC$

$$E_i^T \leq h^2 \left[z^{i-2} \alpha_1 + z^{i-3} \alpha_2 + \dots + z \alpha_{i-2} + \alpha_{i-1} \right]$$

if $|\alpha_i| \leq M$ then

$$\begin{aligned} E_i^T &\leq h^2 M \left[z^{i-2} + z^{i-3} + \dots + z + 1 \right] = h^2 M \left(\frac{z^{i-1} - 1}{z - 1} \right) \\ &= h^2 M \left(\frac{(1 + hC)^{i-1} - 1}{(1 + hC) - 1} \right) \\ &= \frac{Mh}{C} \left((1 + hC)^{i-1} - 1 \right) \end{aligned}$$

- Although we cannot find the exact value of E_i^T , but we can estimate the error bound

$$\text{Since } e^{hC} = 1 + hC + \frac{(hC)^2}{2} + \frac{(hC)^3}{3!} + \dots$$

$$z = 1 + hC \leq e^{hC} \quad \text{or} \quad z^{i-1} = (1 + hC)^{i-1} \leq e^{hC(i-1)}$$

$$\text{so, } E_i^T \leq \frac{hM}{C} [e^{hC(i-1)} - 1]$$

- The last data point is $N+1$ - Accumulated error

$$\begin{aligned} E_{N+1}^T &\leq \frac{hM}{C} [e^{hCN} - 1] \quad , \quad N = \frac{x_{N+1} - x_1}{h} \\ &= \frac{hM}{C} [e^{C[x_{N+1} - x_1]} - 1] \Rightarrow \text{order of } h \end{aligned}$$

➔ Local Truncation error $O(h^2)$

➔ Global Truncation error $O(h)$